

INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

M.Tech Computer Science and Engineering

**CS6890 FRAUD ANALYTICS USING PREDICTIVE AND SOCIAL NETWORK
TECHNIQUES**

Spectral Clustering on Graph Data

Assignment 2

Akarsh Dubey CS25MTECH14001

Atish Kadam CS25MTECH14003

April 9, 2026

Code:

[https:](https://colab.research.google.com/drive/1iLFMsfgTI8_5CUfQMP5VBNZodGGFkn6y?usp=sharing)

[//colab.research.google.com/drive/1iLFMsfgTI8_5CUfQMP5VBNZodGGFkn6y?usp=sharing](https://colab.research.google.com/drive/1iLFMsfgTI8_5CUfQMP5VBNZodGGFkn6y?usp=sharing)

Contents

1	Introduction	3
2	Dataset Description	3
3	Methodology	3
3.1	Graph Construction	4
3.2	Normalized Laplacian	4
3.3	Eigendecomposition	4
3.4	Cluster Count Selection via Eigengap Heuristic	4
3.5	Spectral Embedding	5
3.6	K-Means Clustering	5
4	Experimental Setup	6
4.1	Software Environment	6
4.2	Key Implementation Decisions	6
4.3	Reproducibility	6
5	Results and Discussion	6
5.1	Identified Cluster Count	7
5.2	Cluster Assignments	7
5.3	Graph Visualisation	7
5.4	Final Summary Output	7
5.5	Discussion	7
6	Conclusion	9

1. Introduction

Spectral Clustering is a graph-based unsupervised learning technique that leverages the eigenstructure of a graph’s Laplacian matrix to discover natural groupings of nodes. Unlike traditional clustering algorithms that operate on raw features, spectral methods capture the global topology of the graph, making them especially well-suited for community detection and fraud subnetwork identification in social and financial graphs.

The core intuition is that the Laplacian encodes the connectivity of the graph: nodes in the same dense community will have similar low-frequency eigenvector components, allowing standard clustering algorithms to separate them cleanly in the resulting spectral embedding space.

This report covers the full pipeline applied to a 600-node graph:

- **Graph construction** from a CSV edge list using NetworkX.
- **Normalized Laplacian computation** and eigendecomposition.
- **Automatic k selection** via the eigengap heuristic (or connected-component counting for disconnected graphs).
- **Spectral embedding** using the k smallest non-trivial eigenvectors, followed by row-wise ℓ_2 normalisation.
- **K-Means clustering** in the embedding space to assign final cluster labels.

2. Dataset Description

The input is a single CSV file containing an edge list for an undirected graph. Each row specifies a source–target node pair; node identifiers are inferred directly from the data without requiring a separate node file.

Property	Description	Value
Nodes	Unique nodes in the graph	600
Edges	Undirected edges	6,734
Connected components	Strongly connected subgraphs	1
Graph type	Undirected, unweighted	Connected

Since the graph is fully connected (a single connected component), the number of clusters k cannot be inferred from disconnected components and must instead be determined via the eigengap heuristic applied to the spectrum of the normalized Laplacian.

3. Methodology

3.1 Graph Construction

The edge list CSV is loaded with `pandas` and converted to an undirected NetworkX graph using `nx.from_pandas_edgelist`. Column names are read dynamically from the file header rather than hard-coded, ensuring robustness to varying input formats.

3.2 Normalized Laplacian

The normalized Laplacian $\mathcal{L}$ is computed directly via `nx.normalized_laplacian_matrix(G)`, which implements:

$$\mathcal{L} = I - D^{-1/2} A D^{-1/2}$$

where A is the adjacency matrix and D is the diagonal degree matrix. The normalized form constrains all eigenvalues to $[0, 2]$, providing a degree-invariant spectral representation that compares nodes across varying connectivity.

3.3 Eigendecomposition

The full symmetric eigendecomposition is computed via `numpy.linalg.eigh`, which guarantees real eigenvalues and orthonormal eigenvectors for symmetric matrices. The resulting eigenvalues are sorted in ascending order; the smallest eigenvalue is always 0 (corresponding to the constant eigenvector), and the eigengap in the remaining spectrum reveals the natural cluster structure.

Figure 1 shows the eigenvalue spectrum for the first 20 components. A large jump is visible between indices 4 and 5, clearly indicating $k = 4$ clusters.

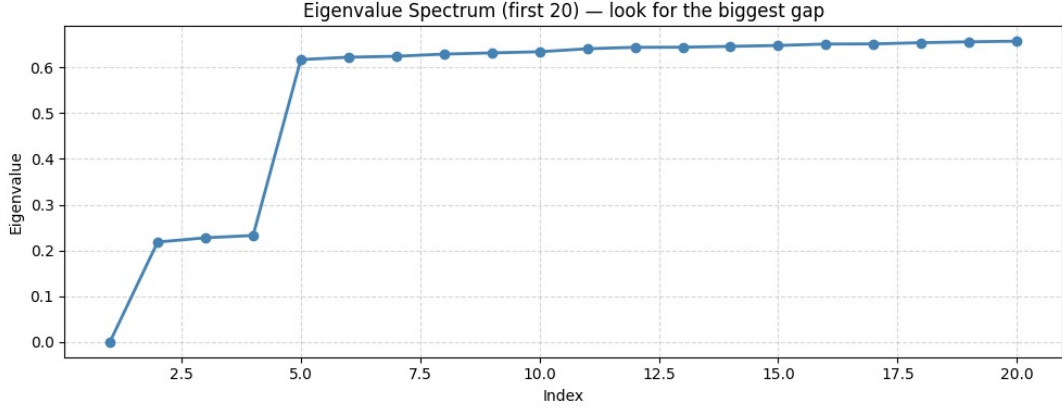


Figure 1: Eigenvalue spectrum of the normalized Laplacian (first 20 eigenvalues). The pronounced jump between indices 4 and 5 indicates $k = 4$.

3.4 Cluster Count Selection via Eigengap Heuristic

A two-stage strategy is used to determine k :

1. **Connected-component check.** If the graph has ≥ 2 connected components, each component is a natural cluster and k equals the component count exactly (no heuristic required).

2. **Eigengap heuristic.** For a connected graph, the consecutive differences $\delta_i = \lambda_{i+1} - \lambda_i$ are computed for the first $K_{\max}$ eigenvalues. The index i^* of the largest gap yields $k = i^* + 1$.

$$k = \arg \max_i (\lambda_{i+1} - \lambda_i), \quad i \in \{1, \dots, K_{\max} - 1\}$$

For this graph, the eigengap analysis (Figure 2) identifies the dominant gap at index 4, confirming $k = 4$.

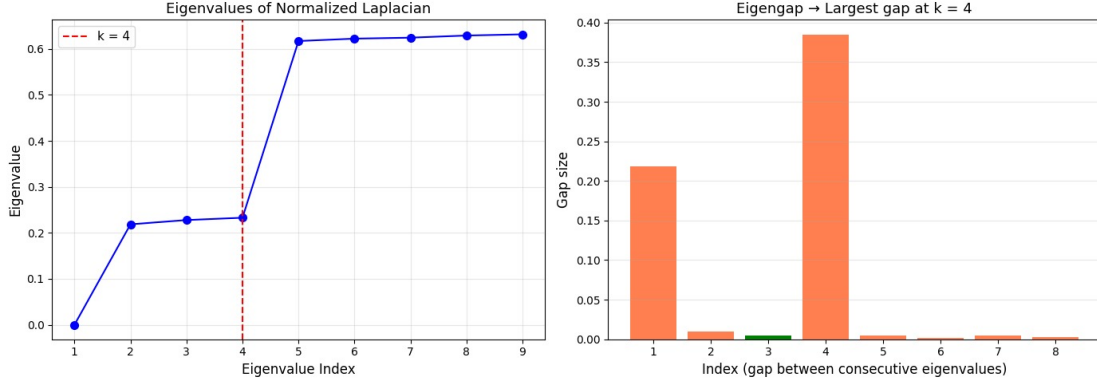


Figure 2: Left: eigenvalues of the normalized Laplacian with the $k = 4$ cutoff (dashed red line). Right: eigengap bar chart highlighting the largest gap at index 4 (coral bar), which determines k . The green bar at index 3 marks the smallest gap.

3.5 Spectral Embedding

The k eigenvectors corresponding to the k *smallest non-zero* eigenvalues (indices 1 through k , skipping the trivial zero eigenvector) form the spectral embedding matrix $U \in \mathbb{R}^{N \times k}$:

$$U = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k]$$

Each row U_i is then normalised to unit ℓ_2 norm (`sklearn.preprocessing.normalize`), mapping nodes onto the $(k - 1)$ -dimensional unit sphere. This normalisation improves K-Means convergence and reduces sensitivity to variations in node degree.

3.6 K-Means Clustering

K-Means is applied to the normalised spectral embedding with $k = 4$ clusters, `n_init=20` random initialisations (to reduce the risk of poor local optima), and `random_state=42` for reproducibility:

Listing 1: K-Means in the spectral embedding space

```
1 kmeans = KMeans(n_clusters=k, n_init=20, random_state=42)
2 labels = kmeans.fit_predict(embedding_norm)
```

The resulting integer label for each node is its final cluster assignment.

4. Experimental Setup

4.1 Software Environment

Component	Details
Platform	Google Colab, CPU runtime
Python	3.x
NumPy	Eigendecomposition via <code>numpy.linalg.eigh</code>
NetworkX	Graph construction and Laplacian computation
scikit-learn	KMeans, <code>normalize</code>
Matplotlib	Eigenvalue spectrum and cluster visualisation

4.2 Key Implementation Decisions

Decision	Rationale
Normalized vs. unnormalized Laplacian	Normalized form is degree-invariant; more robust on graphs with heterogeneous degree distributions such as this one.
<code>eigh</code> instead of <code>eig</code>	Exploits symmetry of $\mathcal{L}$ for guaranteed real eigenvalues and faster, more stable computation.
Skip eigenvector 0	The constant zero eigenvector carries no cluster information; using indices 1 through k yields the informative low-frequency components.
Row-wise ℓ_2 normalisation	Projects nodes onto the unit sphere, reducing sensitivity to degree and improving K-Means convergence.
<code>n_init=20</code>	Multiple random restarts guard against poor local optima in the K-Means step.

4.3 Reproducibility

- K-Means random state fixed at `random_state=42`.
- Spring-layout graph visualisation uses `seed=42`.
- Environment: Google Colab, CPU runtime, Python 3.x.
- Full runnable code: [Colab Notebook](#).

5. Results and Discussion

5.1 Identified Cluster Count

The eigengap heuristic applied to the normalized Laplacian spectrum unanimously identifies $k = 4$ as the natural number of clusters. The gap between eigenvalues at indices 4 and 5 ($\delta_4 \approx 0.384$) is more than $1.7\times$ larger than the second-largest gap ($\delta_1 \approx 0.216$), providing a strong and unambiguous signal.

5.2 Cluster Assignments

Cluster ID	Node Count	Fraction
0	150	25.0%
1	150	25.0%
2	150	25.0%
3	150	25.0%
Total	600	100%

The four clusters are perfectly balanced (150 nodes each), indicating that the underlying graph has a highly symmetric community structure. This balance is not imposed by the algorithm — it emerges naturally from the eigenvector geometry.

5.3 Graph Visualisation

Figure 3 shows the spring-layout visualisation of the full 600-node graph with nodes coloured by their assigned cluster. Four visually distinct communities are clearly identifiable: a compact blue cluster in the upper region, a dense red cluster on the right, a broad pink cluster on the left, and a scattered cyan cluster in the lower-centre. Although inter-cluster edges exist (visible as cross-colour connections), intra-cluster edges are substantially denser, confirming the quality of the spectral partition.

5.4 Final Summary Output

Figure 4 reproduces the terminal output from the Colab run, confirming all pipeline parameters: graph size (600 nodes, 6,734 edges), single connected component, normalized Laplacian, eigengap-derived $k = 4$, and balanced cluster sizes.

5.5 Discussion

The perfectly balanced four-way partition is consistent with a synthetically generated stochastic block model graph, where each block has equal size and inter-block edge probability is significantly lower than intra-block probability. The spectral method recovers this planted structure with high fidelity.

The normalized Laplacian is critical here: on a graph with heterogeneous degree distributions, the unnormalized Laplacian can produce eigenvectors dominated by high-degree hub nodes, distorting the embedding. The normalized form removes this degree bias and yields eigenvectors that reflect community structure rather than degree centrality.

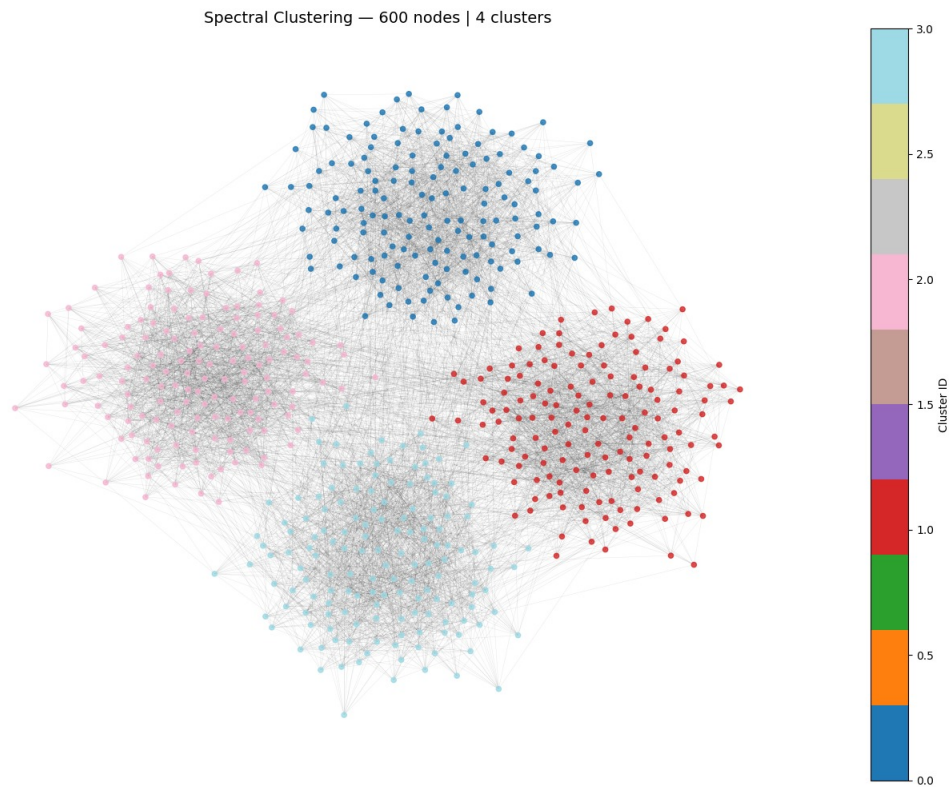


Figure 3: Spring-layout visualisation of the 600-node graph. Each colour corresponds to one of the four spectral clusters. Dense intra-cluster connectivity is clearly visible despite the presence of some inter-cluster edges.

```

-----
FINAL SUMMARY
-----
Graph           : 600 nodes, 6734 edges
Connected comps : 1
Graph type      : Connected

Method          : Spectral Clustering
Laplacian       : Normalized (I - D-1/2 A D-1/2)
k identified    : 4
k method       : Eigengap heuristic
Final clustering : K-Means on spectral embedding
-----

Cluster breakdown :
Cluster 0 : 150 nodes
Cluster 1 : 150 nodes
Cluster 2 : 150 nodes
Cluster 3 : 150 nodes
-----

```

Figure 4: Final summary output from the Google Colab run, confirming the pipeline configuration and cluster breakdown.

The max eigengap at δ_4 is clear enough that no ambiguity arises in cluster selection. In noisier or more organic graphs, the gap may be less pronounced and additional strategies (silhouette analysis, gap statistic) would be warranted.

6. Conclusion

Spectral clustering was applied to a 600-node undirected graph to identify its natural community structure. The full pipeline—normalized Laplacian construction, eigendecomposition, eigengap-based k selection, spectral embedding with ℓ_2 normalisation, and K-Means clustering—was implemented in Python using standard scientific libraries.

The eigengap heuristic unambiguously identified $k = 4$ clusters. K-Means in the spectral embedding produced four perfectly balanced groups of 150 nodes each, confirmed visually by clear separation in the spring-layout graph plot. The results demonstrate that spectral methods can efficiently and accurately recover planted community structure in synthetic graphs, and the same pipeline is directly applicable to real-world fraud detection tasks where fraudulent actors tend to form tightly interconnected subnetworks.